

# CAHIER DES CHARGES TECHNIQUE

*Système de Gestion d'École Primaire (SGEP)*

Version 3.0 - Mai 2026  
Confidentiel - Usage Interne

# CAHIER DES CHARGES TECHNIQUE : SYSTÈME DE GESTION D'ÉCOLE PRIMAIRE (SGEP)

## 1. INTRODUCTION ET CONTEXTE DU PROJET

### 1.1 Objet du Document

Ce document constitue le Cahier des Charges Technique (CDC) détaillé du Système de Gestion d'École Primaire (SGEP). Il a pour objectif de définir l'architecture logicielle, les spécifications fonctionnelles et non fonctionnelles, le modèle de données, les contrats d'API, ainsi que les règles de sécurité et de déploiement. Ce CDC servira de référence principale pour les équipes de développement backend et les parties prenantes du projet.

### 1.2 Contexte et Justification

Le secteur de l'éducation primaire fait face à des défis croissants en matière de gestion administrative et pédagogique. Les processus manuels actuels entraînent des inefficacités, des erreurs et une communication fragmentée. Le SGEP vise à numériser et centraliser ces opérations, offrant une solution robuste et évolutive. La particularité de ce système réside dans son approche backend-centric, où la majorité des traitements complexes et des interactions se déroulent côté serveur, avec une interface utilisateur simplifiée pour les rôles administratifs et une consultation limitée pour les parents.

### 1.3 Périmètre Fonctionnel Global

Le système SGEP couvrira les domaines fonctionnels majeurs suivants, avec une emphase sur la gestion administrative et financière, et un accès contrôlé pour les parents :

- **Gestion des Élèves** : Inscription, suivi scolaire, informations personnelles et médicales.
- **Gestion Pédagogique** : Saisie des notes et des absences (par l'administration), calculs de moyennes et de rangs.
- **Gestion Financière** : Facturation, suivi des paiements, gestion des frais scolaires.
- **Gestion des Utilisateurs et Rôles** : Super Administrateur, Comptable, Parents.
- **Reporting et Exports** : Génération de bulletins PDF, rapports financiers Excel.

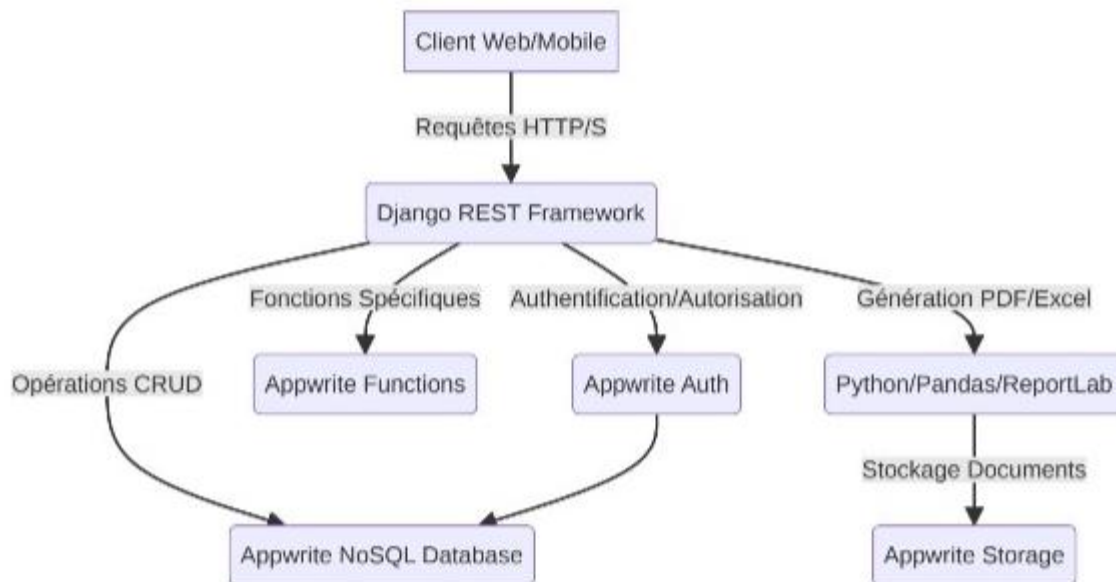
### 1.4 Hypothèses et Contraintes Clés

- **Absence de Portail Enseignant** : Les enseignants n'auront pas d'accès direct à l'application. La saisie des notes et le report des absences seront effectués par le personnel administratif.
- **Base de Données NoSQL** : Utilisation d'Appwrite comme base de données principale pour sa flexibilité et sa performance avec des données non structurées ou semi-structurées.
- **Backend Python avec Django** : Le framework Django sera utilisé pour structurer la logique métier, les API REST et l'intégration avec Appwrite.
- **Génération de Documents** : Les exports PDF et Excel seront gérés côté backend via Python (ReportLab/FPDF) et Pandas.
- **Gestion des Comptes Parents** : Les comptes parents sont créés lors de l'inscription de l'élève et peuvent être suspendus en cas de non-renouvellement annuel.
- **Sécurité** : Protection des données sensibles, authentification robuste et gestion fine des autorisations.

## 2. ARCHITECTURE TECHNIQUE DÉTAILLÉE

### 2.1 Vue d'Ensemble de l'Architecture

L'architecture du SGEP est conçue pour être modulaire, performante et évolutive. Elle combine la puissance de Django pour la logique métier et l'exposition d'API, avec la flexibilité et la scalabilité d'Appwrite pour la persistance des données et l'authentification.



#### Description des Composants :

- **Client Web/Mobile** : Représente les interfaces utilisateur (non détaillées dans ce CDC) qui consomment les API exposées par le backend Django.
- **Django REST Framework (DRF)** : Le cœur du backend, gérant les requêtes HTTP, la logique métier, la validation des données et l'orchestration des interactions avec Appwrite et les modules de génération de documents.
- **Appwrite Auth** : Gère l'authentification des utilisateurs (Super Administrateur, Comptable, Parents) et la gestion des sessions. Django s'intégrera avec Appwrite pour la vérification des tokens.
- **Appwrite NoSQL Database** : La base de données principale pour stocker toutes les collections de données (élèves, notes, finances, etc.). Sa nature NoSQL offre une grande flexibilité pour les schémas de données évolutifs.
- **Appwrite Functions** : Utilisées pour des traitements asynchrones ou des déclencheurs spécifiques (ex: notification après un paiement, mise à jour de statut).
- **Python/Pandas/ReportLab** : Modules Python dédiés à la génération de rapports complexes. Pandas sera utilisé pour la manipulation et l'exportation de données vers Excel, tandis que ReportLab ou FPDF sera privilégié pour la création de documents PDF structurés (bulletins, reçus).

- **Appwrite Storage** : Solution de stockage d'objets pour archiver les documents générés (PDF, Excel) et d'autres fichiers (photos d'élèves, justificatifs).

## 2.2 Stack Technologique Détaillé

| Couche / Composant       | Technologie           | Version | Rôle et Intégration Spécifique                                                                       |
|--------------------------|-----------------------|---------|------------------------------------------------------------------------------------------------------|
| Backend Framework        | Django                | 5.x     | Logique métier principale, gestion des URL, vues, middleware.                                        |
| API Framework            | Django REST Framework | 3.x     | Exposition des endpoints RESTful, sérialisation, permissions.                                        |
| Base de Données (NoSQL)  | Appwrite              | 1.x     | Persistance des données (collections), gestion des utilisateurs et des droits d'accès aux documents. |
| Langage de Programmation | Python                | 3.11+   | Développement des services backend, scripts de traitement, fonctions Appwrite.                       |
| Génération PDF           | ReportLab             | Latest  | Création de bulletins scolaires, reçus de paiement avec mise en page complexe.                       |
| Export Exce              | Pandas                | Latest  | Manipulation de données pour les rapports financiers et exports XLSX.                                |
| Authentification         | Appwrite Auth         | 1.x     | Gestion des sessions utilisateur, intégration via des tokens JWT ou sessions Appwrite.               |
| Stockage Fichiers        | Appwrite Storage      | 1.x     | Stockage des documents générés (PDF, Excel), photos d'élèves.                                        |
| Tâches Asynchrones       | Celery (avec Redis)   | 5.x     | Gestion des tâches de fond (envoi de notifications, génération de rapports lourds).                  |

## 2.3 Modèle de Données (Appwrite Collections)

Le modèle de données est basé sur des collections Appwrite, offrant une grande flexibilité. Chaque collection est un ensemble de documents JSON. Les relations entre les entités seront gérées par des identifiants de documents.

### 2.3.1 Collections Principales

- **`users` (Appwrite)** : Gère les informations d'authentification et les rôles (SuperAdmin, Comptable, Parent). Chaque utilisateur Appwrite sera lié à un profil spécifique dans Django.
- **`students`** :

``studentId` (UUID)` : Identifiant unique de l'élève.

``firstName`, `lastName`, `birthDate`, `gender``.

``matricule` (String, Unique)` : Numéro d'immatriculation.

``medicalNotes` (Text)` : Informations médicales (allergies, traitements).

``isActive` (Boolean)` : Statut actif/inactif dans l'école.

``classId` (Reference)` : Lien vers la collection ``classes``.

``parentIds` (Array of References)` : Liens vers les documents ``parents``.

- **``parents`` :**

``parentId` (UUID)` : Identifiant unique du parent.

``userId` (Reference)` : Lien vers l'utilisateur Appwrite correspondant.

``firstName`, `lastName`, `email`, `phone``.

``studentIds` (Array of References)` : Liens vers les documents ``students``.

``accountStatus` (Enum: `ACTIVE`, `SUSPENDED`)` : Statut du compte parent.

``lastRenewalDate` (Date)` : Date du dernier renouvellement d'inscription.

- **``classes`` :**

``classId` (UUID)` : Identifiant unique de la classe.

``name` (String)` : Ex:

CP-A, CE1-B.

``academicYearId` (Reference)` : Lien vers la collection ``academicYears``.

``teacherId` (Reference)` : Lien vers la collection ``teachers`` (pour l'enseignant principal).

- **``academicYears`` :**

``yearId` (UUID)` : Identifiant unique de l'année scolaire.

``startDate`, `endDate``.

``current` (Boolean)` : Indique l'année scolaire en cours.

- **``grades`` :**

``gradeId` (UUID)` : Identifiant unique de la note.

``studentId` (Reference)` : Élève concerné.

``subject` (String)` : Matière (ex: Mathématiques, Français).

``score` (Float)` : Note obtenue.

``coefficient` (Integer)` : Coefficient de la matière.

``period` (String)` : Période d'évaluation (ex: Trimestre 1, Séquence 2).

``recordedBy` (Reference)` : Utilisateur ayant enregistré la note.

``recordedAt` (Timestamp)`.

- **``attendance`` :**

``attendanceId` (UUID)` : Identifiant unique de l'enregistrement d'absence.

``studentId` (Reference)` : Élève concerné.

``date` (Date)` : Date de l'absence.

``status` (Enum: `PRESENT`, `ABSENT`, `LATE`, `JUSTIFIED_ABSENT`)`.

``reason` (Text)` : Motif de l'absence.

``recordedBy` (Reference)` : Utilisateur ayant enregistré l'absence.

``recordedAt` (Timestamp)`.

- **``invoices`` :**

``invoiceId` (UUID)` : Identifiant unique de la facture.

``studentId` (Reference)` : Élève concerné.

``amount` (Float)` : Montant total.

``dueDate` (Date)` : Date d'échéance.

``status` (Enum: `PENDING`, `PAID`, `OVERDUE`)`.

``items` (Array of Objects)` : Détail des frais (type, montant).

- **``payments`` :**

``paymentId` (UUID)` : Identifiant unique du paiement.

``invoiceId` (Reference)` : Facture associée.

``amountPaid` (Float)` : Montant payé.

``paymentDate` (Date)`.

``method` (String)` : Mode de paiement (espèces, chèque, virement).

``recordedBy` (Reference)` : Utilisateur ayant enregistré le paiement.

### 2.3.2 Relations et Intégrité des Données

Bien qu'Appwrite soit NoSQL, des mécanismes seront mis en place pour assurer la cohérence des données :

- **Références de Documents** : Utilisation des IDs de documents pour lier les collections entre elles (ex: ``studentId`` dans ``grades``).
- **Règles de Validation** : Définition de règles de validation au niveau des collections Appwrite pour garantir l'intégrité des données (ex: ``score`` entre 0 et 20).
- **Hooks et Fonctions Appwrite** : Utilisation de fonctions Appwrite pour maintenir la cohérence lors des opérations CRUD complexes (ex: mise à jour du statut ``accountStatus`` d'un parent lors de la réinscription d'un élève).

## 3. GESTION DES UTILISATEURS ET RÔLES

### 3.1 Rôles et Permissions Détaillés

Le système implémente un modèle de contrôle d'accès basé sur les rôles (RBAC) avec des permissions granulaires gérées par Django et Appwrite.

| Rôle                 | Description                                                                                             | Permissions Clés (Exemples)                                                                                                                                                |
|----------------------|---------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Super Administrateur | Accès complet au système. Gère la configuration, les utilisateurs, les élèves, les notes, les finances. | Création/Modification/Suppression de tous les documents. Accès aux paramètres système. Gestion des comptes parents.                                                        |
| Comptable            | Gère toutes les opérations financières.                                                                 | Création/Modification de factures et paiements. Génération de rapports financiers. Accès en lecture aux données élèves et parents (limitées aux informations financières). |
| Parent               | Accès en consultation aux informations de ses enfants.                                                  | Lecture des notes, absences, bulletins, factures de ses enfants. Pas de modification.                                                                                      |

### 3.2 Cycle de Vie et Suspension des Comptes Parents

Le compte parent est un élément central pour la communication et le suivi scolaire. Son statut est directement lié à la situation administrative de l'élève.

#### 3.2.1 Création du Compte Parent

- Lors de l'inscription d'un nouvel élève par le Super Administrateur, un compte parent est automatiquement créé dans Appwrite (si non existant) et lié à l'élève.
- Le parent reçoit des identifiants de connexion initiaux pour accéder à son espace.

#### 3.2.2 Renouvellement Annuel et Suspension

- Chaque année scolaire, un processus de réinscription est initié par l'administration.
- Si un parent ne renouvelle pas l'inscription de son enfant pour l'année en cours, le système déclenche la suspension du compte parent.
- **Mécanisme de Suspension** :

*Un script Python (via une tâche Celery) vérifie périodiquement le statut de réinscription des élèves.*

*Si un élève n'est pas réinscrit, le `accountStatus` du parent associé est mis à jour à `SUSPENDED` dans la collection `parents` d'Appwrite.*

*Les permissions d'accès aux documents (notes, absences, factures) pour ce parent sont révoquées ou restreintes via les règles de sécurité d'Appwrite et la logique de Django.*

- **Fonctionnalités Suspendues :**

*Accès aux bulletins scolaires et historiques de notes.*

*Consultation des absences détaillées.*

*Accès à la messagerie interne (si implémentée).*

*Consultation des factures et paiements.*

- **Réactivation :** Le compte est réactivé automatiquement dès que l'élève est réinscrit pour l'année en cours.

## 4. MODULES FONCTIONNELS DÉTAILLÉS

### 4.1 Module Élèves et Inscriptions

Ce module est la pierre angulaire du système, gérant toutes les informations relatives aux élèves.

#### 4.1.1 Fonctionnalités Clés

- **Gestion des Fiches Élèves :** Création, modification, consultation des informations personnelles (nom, prénom, date de naissance, genre, matricule), coordonnées des parents, contacts d'urgence, informations médicales (allergies, traitements).
- **Inscription et Réinscription :** Processus guidé pour l'enregistrement de nouveaux élèves et la réinscription annuelle. Liaison automatique avec les comptes parents.
- **Affectation aux Classes :** Attribution manuelle ou semi-automatique des élèves à une classe pour une année scolaire donnée.
- **Historique Scolaire :** Suivi des classes fréquentées, des années scolaires, des promotions et des redoublements.
- **Recherche et Filtrage :** Outils de recherche multicritères pour retrouver rapidement les élèves (par nom, classe, statut, etc.).

#### 4.1.2 Intégration Django-Appwrite

- **Modèles Django :** Des modèles Django (ex: `Student`, `Guardian`) seront définis pour représenter les données des élèves et des parents, avec des champs correspondant aux attributs des collections Appwrite.
- **Serializers DRF :** Utilisation de Django REST Framework pour sérialiser/désérialiser les données entre les requêtes API et les collections Appwrite.
- **Opérations CRUD :** Les vues Django effectueront les opérations CRUD sur les collections Appwrite via le SDK Python d'Appwrite.



## 4.2 Module Pédagogique (Gestion Centralisée)

Ce module gère les aspects liés aux notes et aux absences, avec une particularité forte : l'absence d'accès direct pour les enseignants.

### 4.2.1 Gestion des Notes et Évaluations

- **Saisie Centralisée** : Le Super Administrateur ou un agent administratif dédié est responsable de la saisie des notes transmises par les enseignants.
- **Pondération Configurable** : Les coefficients des matières et des périodes d'évaluation sont configurables par l'administration.
- **Moteur de Calcul Python** : Un service Python en backend est responsable des calculs suivants :

**Moyenne par Matière** : Calculée en fonction des notes et de leurs coefficients.

**Moyenne Générale** : Calculée à partir des moyennes par matière.

**Rang de l'Élève** : Détermination du classement de l'élève au sein de sa classe pour une période donnée, en gérant les ex-aequo.

- **Appréciations** : Possibilité d'ajouter des appréciations textuelles par matière et une appréciation générale.

### 4.2.2 Gestion des Absences

- **Report Manuel par les Enseignants** : Les enseignants reportent les absences (présent, absent, retard, absent justifié) sur des fiches physiques ou via un processus hors-système.
- **Saisie Administrative** : Le Super Administrateur ou un agent administratif saisit ces informations dans le système.
- **Suivi et Historique** : Enregistrement de la date, du statut, du motif et du justificatif (si applicable) de chaque absence. Historique consultable par l'administration et les parents (pour leurs enfants).
- **Alertes Automatiques** : Possibilité d'envoyer des notifications automatiques aux parents en cas d'absence non justifiée (via Appwrite Functions ou Celery).

## 4.3 Module Finance et Recouvrement

Ce module est dédié à la gestion des frais scolaires, à la facturation et au suivi des paiements.

### 4.3.1 Fonctionnalités Clés

- **Configuration des Frais** : Définition des différents types de frais (inscription, scolarité, cantine, activités) et de leurs montants.
- **Facturation Automatique** : Génération de factures pour les élèves en fonction de leur inscription et des frais configurés.
- **Enregistrement des Paiements** : Saisie des paiements reçus (espèces, chèque, virement, mobile money). Génération automatique de reçus PDF.
- **Suivi des Impayés** : Tableau de bord des factures en attente de paiement, avec des indicateurs d'ancienneté.
- **Relances Automatiques** : Envoi de rappels aux parents pour les factures impayées (via email/SMS, géré par Celery).

#### 4.3.2 Exports Financiers (Pandas)

- **Journal de Caisse** : Exportation détaillée des transactions financières (recettes et dépenses) sur une période donnée au format Excel (XLSX) via Pandas.
- **Rapports de Recouvrement** : Rapports synthétiques sur le taux de recouvrement, les impayés par classe ou par type de frais.
- **Compatibilité Comptable** : Les exports sont conçus pour être facilement intégrables dans des logiciels de comptabilité tiers.

## 5. SPÉCIFICATIONS DES ENDPOINTS API (DJANGO REST FRAMEWORK)

Les API sont le point d'accès principal aux fonctionnalités du SGEP. Elles sont conçues pour être RESTful, sécurisées et bien documentées.

### 5.1 Conventions Générales

- **Base URL** : `/api/v1/`
- **Format** : JSON pour les requêtes et les réponses.
- **Authentification** : Basée sur les sessions Appwrite ou des tokens JWT générés par Django après authentification via Appwrite.
- **Permissions** : Gérées par DRF (Django REST Framework) en conjonction avec les rôles Appwrite.
- **Pagination, Filtrage, Tri** : Standard DRF (ex: `?page=X&page\_size=Y`, `?status=pending`, `?ordering=-date`).

### 5.2 Endpoints par Module

#### 5.2.1 Authentification et Utilisateurs

- `POST /api/v1/auth/login/` : Authentification d'un utilisateur (Super Admin, Comptable, Parent) via Appwrite. Retourne un token de session ou JWT.
- `POST /api/v1/auth/logout/` : Déconnexion de l'utilisateur.
- `GET /api/v1/users/me/` : Récupère les informations de l'utilisateur connecté.
- `PATCH /api/v1/parents/{id}/status/` : (Super Admin) Met à jour le statut de suspension d'un compte parent.

#### 5.2.2 Élèves et Classes

- `GET /api/v1/students/` : Liste des élèves (filtrable par classe, année scolaire, statut). Rôle: Super Admin, Comptable (lecture limitée).
- `POST /api/v1/students/` : Création d'un nouvel élève (déclenche la création/liaison du compte parent). Rôle: Super Admin.
- `GET /api/v1/students/{id}/` : Détail d'un élève. Rôle: Super Admin, Comptable (lecture limitée), Parent (pour son enfant).
- `PUT /api/v1/students/{id}/` : Mise à jour des informations d'un élève. Rôle: Super Admin.
- `POST /api/v1/students/{id}/enroll/` : Inscription/Réinscription d'un élève à une classe. Rôle: Super Admin.
- `GET /api/v1/classes/` : Liste des classes. Rôle: Super Admin.

#### 5.2.3 Notes et Absences

- `POST /api/v1/grades/` : Saisie d'une note pour un élève. Rôle: Super Admin.
- `GET /api/v1/grades/{student\_id}/` : Récupère les notes d'un élève. Rôle: Super Admin, Parent (pour son enfant).

- `POST /api/v1/attendance/` : Enregistrement d'une absence. Rôle: Super Admin.
- `GET /api/v1/attendance/{student\_id}/` : Récupère l'historique des absences d'un élève. Rôle: Super Admin, Parent (pour son enfant).
- `GET /api/v1/reports/bulletin/{student\_id}/` : **Génère et retourne le bulletin scolaire au format PDF.** Rôle: Super Admin, Parent (pour son enfant).

#### 5.2.4 Finance

- `GET /api/v1/invoices/` : Liste des factures. Rôle: Super Admin, Comptable, Parent (pour ses factures).
- `POST /api/v1/invoices/` : Création d'une facture. Rôle: Super Admin, Comptable.
- `POST /api/v1/payments/` : Enregistrement d'un paiement. Rôle: Comptable.
- `GET /api/v1/payments/{id}/receipt/` : **Génère et retourne le reçu de paiement au format PDF.** Rôle: Comptable, Parent (pour ses reçus).
- `GET /api/v1/finance/export/excel/` : **Génère et retourne un rapport financier au format XLSX (Excel).** Rôle: Super Admin, Comptable.

## 6. GÉNÉRATION DE DOCUMENTS ET EXPORTS

La capacité à générer des documents officiels et des rapports structurés est une exigence clé du SGEP. Ces opérations sont centralisées dans le backend Python.

### 6.1 Bulletins Scolaires (PDF)

- **Technologie** : ReportLab ou FPDF (bibliothèques Python).
- **Contenu** : Le bulletin inclura l'en-tête de l'école, les informations de l'élève, les notes par matière avec coefficients, les moyennes par matière et générale, le rang dans la classe, et les appréciations. La mise en page sera conforme aux standards académiques.
- **Processus** : Lors d'une requête à l'endpoint `/api/v1/reports/bulletin/{student\_id}/`, le backend Django récupère les données nécessaires (notes, appréciations) depuis Appwrite, effectue les calculs via Python, puis utilise ReportLab/FPDF pour générer le PDF en mémoire et le renvoyer au client.

### 6.2 Reçus de Paiement (PDF)

- **Technologie** : ReportLab ou FPDF.
- **Contenu** : Informations sur l'école, le payeur, le montant, la date, le mode de paiement et la référence de la facture.
- **Processus** : Similaire aux bulletins, généré à la demande via l'endpoint `/api/v1/payments/{id}/receipt/`.

### 6.3 Rapports Financiers (Excel - XLSX)

- **Technologie** : Pandas (bibliothèque Python).
- **Contenu** : Rapports personnalisables tels que le journal de caisse, la liste des impayés, le bilan des recettes. Les données seront agrégées et formatées pour une analyse facile.
- **Processus** : L'endpoint `/api/v1/finance/export/excel/` déclenchera un script Python qui interrogera Appwrite pour les données financières, utilisera Pandas pour les structurer dans un DataFrame, puis exportera ce DataFrame en fichier XLSX. Le fichier sera ensuite renvoyé au client ou stocké temporairement dans Appwrite Storage.

## 7. SÉCURITÉ ET CONFORMITÉ

La sécurité des données des élèves et des informations financières est primordiale.

## 7.1 Authentification et Autorisation

- **Authentification Appwrite** : Les utilisateurs s'authentifient via Appwrite, qui gère les identifiants et les sessions.
- **Tokens JWT/Sessions Django** : Django utilisera des tokens JWT ou des sessions pour maintenir l'état d'authentification et vérifier les permissions.
- **Permissions Granulaires** : Utilisation des permissions de Django REST Framework (DRF) et des règles de sécurité d'Appwrite pour contrôler l'accès aux ressources au niveau le plus fin (ex: un parent ne peut voir que les données de ses enfants).

## 7.2 Protection des Données

- **HTTPS** : Toutes les communications entre le client et le backend seront chiffrées via HTTPS.
- **Chiffrement des Données Sensibles** : Les données médicales ou financières sensibles seront chiffrées au repos dans Appwrite (si supporté nativement ou via une couche d'application).
- **Politique de Mots de Passe** : Exigence de mots de passe forts (longueur minimale, caractères spéciaux, etc.).

## 7.3 Audit et Traçabilité

- **Journalisation des Actions** : Toutes les opérations critiques (création/modification d'élève, saisie de note, enregistrement de paiement) seront journalisées avec l'utilisateur, la date et l'heure.
- **Historique des Modifications** : Appwrite peut être configuré pour conserver l'historique des modifications de documents, assurant une traçabilité complète.

# 8. DÉPLOIEMENT ET MAINTENANCE

## 8.1 Environnements

- **Développement** : Environnement local Docker Compose (Django, Appwrite, Redis).
- **Staging** : Environnement de pré-production pour les tests d'intégration et la validation.
- **Production** : Environnement robuste et sécurisé, avec des mécanismes de haute disponibilité.

## 8.2 Stratégies de Déploiement

- **Conteneurisation** : Utilisation de Docker pour l'ensemble des services (Django, Celery, etc.) afin d'assurer la portabilité et la reproductibilité.
- **CI/CD** : Mise en place d'une chaîne d'intégration et de déploiement continu pour automatiser les tests et les déploiements.

## 8.3 Sauvegarde et Récupération

- **Sauvegarde Appwrite** : Stratégie de sauvegarde régulière des collections et des fichiers Appwrite.
- **Sauvegarde Base de Données Django** : Si Django utilise une base de données relationnelle secondaire pour certaines données, celle-ci sera également sauvegardée.
- **Plan de Reprise d'Activité (PRA)** : Définition d'un RTO (Recovery Time Objective) et RPO (Recovery Point Objective) pour garantir la continuité de service.

# 9. GLOSSAIRE

- **Appwrite** : Backend-as-a-Service open-source offrant des services d'authentification, de base de données NoSQL, de stockage et de fonctions.
- **CDC** : Cahier des Charges Technique.

- **Django** : Framework web Python de haut niveau qui encourage le développement rapide et la conception propre et pragmatique.
- **DRF** : Django REST Framework, une boîte à outils flexible pour construire des API web.
- **NoSQL** : Base de données non relationnelle, offrant une flexibilité de schéma et une scalabilité horizontale.
- **Pandas** : Bibliothèque Python pour la manipulation et l'analyse de données.
- **PDF** : Portable Document Format, format de fichier pour la présentation de documents.
- **RBAC** : Role-Based Access Control, méthode de restriction de l'accès au système basée sur les rôles des utilisateurs.
- **ReportLab / FPDF** : Bibliothèques Python pour la génération de documents PDF.
- **SGEP** : Système de Gestion d'École Primaire.
- **XLSX** : Format de fichier pour les feuilles de calcul Microsoft Excel.